

FULL-DAY WORKSHOP

Agentic End-to-End Developer Experience

Data Engineering from sources to production. Real-time ingestion, storage with governance, semantic layers, and agentic deployment—all in one day.

Prepared for **Workshop Participants**, Data Engineering & Analytics



TODAY

Building the complete data layer. From **sources to users.**

Six hands-on modules. Build real code in your workspace. Leave with a working data pipeline, governance wired in, and the playbook for production.

01

09:00-10:30

Ingest

Real-time Kafka → Zerobus → Databricks. SDP, expectations, DLT metadata.

02

10:45-11:45

Storage & Governance

Delta Lake + Unity Catalog. Domains, classification, ABAC, lineage.

03

12:00-13:00

LUNCH

Break. Talk to teammates.

04

13:00-14:15

Transactional & Analytical

Lakebase, real-time CRUD, serverless warehousing.

05

14:30-15:30

Semantic

Metric views, domains, certification. Genie ontology.

06

15:45-17:00

Product to Production

PR gates, Asset Bundles, deploy & schedule. Ship it.

VISION

Where agentic development **goes.**

Real-time sales and pricing command center. Built on the same platform layer you build today.

Store Pulse

LIVE DASHBOARD

Real-time sales by product, store, and price point. Auto-flagged anomalies.

Price Intelligence

GENIE AGENT

Ask questions: 'Which stores are underselling Tropicana?' Natural language, no SQL.

Markdown Forecast

ACTION

Lakebase forms. Predict impact of price changes. Capture decisions in the data.

Inventory Command

REAL-TIME CRUD

Update stock via app. Audit trail in UC. Available instantly in analytics.

THE STACK

Five layers. Built today. Production ready.

Each layer is independent. Mix and match what you need.

5

ML & AI

Feature pipelines, Genie agents, forecasting models. Seat it on the semantic layer.

4

Semantic

Metric views, domains, certified tables. Single source of truth for metrics and dimensions.

3

Transactional & Analytical

Real-time CRUD via Lakebase. Serverless SQL for analytics. One lakehouse.

2

Storage & Governance

Delta + UC + Iceberg. Tag data, enforce access, track lineage. Built-in audit.

1

Ingest

Real-time Kafka/Zerobus or batch. SDP + expectations. Land and validate cleanly.

REAL USE CASES

The three anchor patterns. Build these **today**.

UC1

Real-time Sales Stream

Source: Kafka → Zerobus

6 events/sec, 50 stores

Ingest: SDP pipeline

Bronze → silver → gold medallion

Serve: Semantic layer

Ask Genie for top items by revenue

UC2

Item Price Updates

Source: Batch SQL extract

Items, categories, current price

Storage: UC table + domains

PII tagged, certified

Action: Lakebase form

Change price, audit captured

UC3

Reference Data (MySQL)

Source: Lakeflow Connect

Sync store locations, hours

Governance: Tags + grants

Row-level access by region

Join: In gold tables

Enriched facts for analytics

HOW IT WORKS

Agentic development. The **loop**.

Every artifact: describe → generate → review → deploy. In your repo. Governed by UC.

Describe

Tell Genie Code what you want.
Screenshot, plain language, example SQL.
No templates.

Generate

Genie Code writes the notebook,
pipeline, app, or schema. Shows you a
plan first.

Review

Read it. Compare to reference. Keep a
human in the loop. Ask for changes.

Deploy

Ship via Asset Bundles or Publish button.
Governed by Databricks IAM. Multi-user
ready.

PART 1 OF 5

Ingest: Real-time Sources

INGEST

From Kafka to gold. In **one pipeline.**

Zerobus → SDP with expectations → medallion. Build the source today.

Zerobus Publisher

SOURCE

Stream events directly from Kafka to Databricks. Authentication via service principal. No Spark jobs, no latency.

SDP + Expectations

VALIDATE

Catch bad data early. Schema, not-null, and custom rules. DLT tracks lineage and metadata.

Medallion

TRANSFORM

Bronze (raw), silver (cleaned), gold (aggregated). Liquid clustering on store + item. Ready for queries.

Build the Ingest Pipeline

1

Deploy Zerobus Publisher

Use Genie Code to scaffold the Python publisher. Stream 6 events/sec to bronze.sales_events.

2

Define Expectations

Schema validation, not-null constraints, timestamp format check. SDP rejects invalid events.

3

Build Silver → Gold

Genie Code creates silver_sales and gold_store_item_daily. Aggregates by store, item, date.

4

Test End-to-End

Publish events. Watch them flow through medallion. Query gold table. Confirm data.

Scaffold the Zerobus Publisher

Build a Python Zerobus publisher for Databricks. It should:

- Use the Zerobus Ingest SDK (pip install databricks-zerobus-ingest-sdk)
- Connect to: `https://your-workspace.zerobus.eastus.azuredatabricks.net`
- Authenticate using service principal (app_id and secret from scope `publix_workshop`)
- Define two Zerobus streams:
 1. `publix_agentic_workshop.bronze.sales_events` (6 Publix products, prices)
 2. `publix_agentic_workshop.bronze.price_updates`
- For 60 seconds, publish synthetic events:
 - 3-6 sales/sec to `sales_events` (random store, product, quantity)
 - Occasional price updates (10% chance per second)
- Use fire-and-forget ingestion (`ingest_record_nowait`)
- Flush and close streams cleanly at the end

Generate production-ready script. Show the plan and code.

PART 2 OF 5

Storage & Governance

GOVERNANCE

Delta + UC. Built-in **audit.**

Three-level namespace, domains, PII tagging, row-level access, lineage. All tracked.

Unity Catalog 3-Level Namespace

STORAGE

catalog.schema.table. Cluster-independent. Managed identity. One source of truth across workspaces.

Domain Tagging & AI Classification

DISCOVERY

Tag tables with business domains (sales, pricing, inventory). AI auto-classifies PII. Keep it findable.

ABAC & AI Gateway

ACCESS

Attribute-based access control. Role-based. Apply rules at read time. Gateway scans queries for violations.

Lineage & Audit

TRACK

See who accessed what, when. Track transformations. Query history. Built into UC.

Register & Tag Data

1

Register Bronze, Silver, Gold Schemas

Create UC schemas in `publix_agentic_workshop`. Grant service principal `CAN_RUN_SQL`.

2

Tag with Domains

`Sales_events` → domain:sales. `Price_updates` → domain:pricing. Use UI or SQL.

3

Walk Lineage in Databricks

Navigate UC → table → Lineage tab. See bronze → silver → gold. Ask Genie about transformations.

4

Set Row-Level Filters

Restrict `store_number` by user region. Apply via ABAC rule. Query returns filtered results.

Unity Catalog Registration & Tagging

```
Generate SQL to register the Publix medallion schemas in Unity Catalog. It should:  
- Create catalog publix_data (if not exists)  
- Create three schemas: bronze, silver, gold  
- Grant CAN_MANAGE_METADATA to service principal your-service-principal  
- Tag each schema with domain: bronze→ingest, silver→transformations, gold→analytics  
- Add description comments  
Show me the DDL and how to apply the tags via API or Databricks UI.
```

→ produces working code + plan

PART 3 OF 5

Transactional & Analytical

REAL-TIME LAKEHOUSE

Lakebase + Serverless. One data **engine**.

CRUD forms in Lakebase. Analytics queries via serverless SQL. No ETL explosion, no dual systems.

Lakebase for Real-Time CRUD

FORMS

SQL forms for updates: price, inventory, store hours. Capture decisions as data. Multi-user safe.

Serverless SQL Warehouse

QUERY

Query gold tables instantly. No cluster to manage. Auto-scales. Separate compute from storage.

LTAP (Low-Latency Analytical Processing)

HYBRID

One query engine. Transactional consistency + analytical scale. Real-time dashboards.

Build a Lakebase Form & Query

1

Create Price Update Form in Lakebase

Form for item_id, new_price, effective_date. Submit writes to price_updates table.

2

Run Serverless SQL Query

Ask serverless WH for 'top 10 items by revenue today'. Runs instantly, no cluster.

3

Verify Consistency

Update price via form. Query updates immediately. Audit trail captured.

4

Build Real-Time Dashboard

SQL dashboard on gold_store_item_daily. Refresh every 5 sec. Lakebase embeds Genie.

Lakebase CRUD Form & Schema

Draft a Lakebase form and SQL schema for price updates. It should:

- Define a price_updates table: item_id, new_price, effective_date, updated_by, updated_at
- Add primary key and NOT NULL constraints
- Create a Lakebase form with dropdowns for item_id (from item_master), date picker, authorization check
- Write the SQL trigger or stored procedure to update the gold_store_item_daily materialized view
- Show me the form config and the DDL.

Assume serverless SQL warehouse your-warehouse.

PART 4 OF 5

Semantic Layer

SEMANTIC

One truth. Metric views. Genie **agents**.

Certify tables. Define metrics. Genie understands the data. Users ask questions, not queries.

Metric Views

DEFINE

SQL-defined measures (sum, avg, count) and dimensions. Genie Code uses these for consistent answers.

Domain Tagging

ORGANIZE

Group tables by business area. Tagging scope Genie's search. Makes discovery easy.

Certification & Comments

TRUST

Mark tables as certified. Add descriptions. UI shows trust badges. Users know what's authoritative.

Genie Agents & Ontology

QUERY

Genie space knows the schema + metrics + domains. Answers questions in natural language. No SQL.

Build Metric View & Genie Space

1

Create Metric View

Define total_revenue (SUM), avg_units (AVG), store_rank by revenue on gold_store_item_daily.

2

Tag and Certify

Mark gold_store_item_daily as Certified. Add domain tags: sales, analytics. Add description.

3

Create Genie Space

Point Genie at the metric view. Upload instructions. Genie knows the schema and metrics.

4

Ask Genie Questions

Try: 'Top 5 stores by revenue this month' and 'Which items are trending?' Genie answers both.

Metric View & Genie Ontology

Generate a metric view and Genie instructions for Store Performance. It should:

- Create metric_store_performance on gold_store_item_daily
- Define measures: total_revenue (SUM), units_sold (SUM), avg_price (AVG)
- Define dimensions: store_number, item_id, item_name, item_category, sales_date
- Write Genie space instructions (< 12 lines):
 - State grain: daily by store and item
 - Explain revenue is pre-aggregated, never double-count
 - List key dimensions
 - Tell Genie to always group by the grain the user asks for
- Provide three sample questions (easy, medium, hard)

Show me the metric view DDL and the Genie instructions.

PART 5 OF 5

Product to Production

DEPLOYMENT

From notebook to production. Asset **Bundles**.

One config file. PR review gates. Deploy to prod with confidence. Schedule jobs, monitor lineage.

Databricks Asset Bundles (DAB)

CONFIG

Single databricks.yml. Define notebooks, pipelines, jobs, clusters, permissions. Version control.

PR Review Gates

QUALITY

Push to GitHub. DAB validates config. Review cost, lineage, permissions. Merge → deploy.

Deploy & Schedule

SHIP

One command: databricks bundle deploy. Pushes to prod workspace. Jobs run on schedule.

Monitor & Lineage

OPERATE

Job history, lineage, cost by pipeline. Built into Databricks. Audit trail complete.

Deploy via Asset Bundles

1**Create databricks.yml**

Define resources: notebooks, DLT pipelines, jobs, service principal permissions.

2**Validate Configuration**

Run databricks bundle validate. Checks syntax, permissions, resource names.

3**Deploy to Dev**

databricks bundle deploy --target dev. Uploads code, creates resources, starts first job.

4**Promote to Prod**

Same config, different target. databricks bundle deploy --target prod. Same guarantees.

Asset Bundle Configuration

Generate a databricks.yml Asset Bundle for the Publix medallion pipeline. It should:

- Define workspace details: host, token (via environment var)
- Define two jobs:
 1. dlt_pipeline: Runs SDP medallion daily at 06:00 UTC
 2. genie_space_sync: Refreshes Genie metric views daily at 07:00 UTC
- Define service principal: your-service-principal with CAN_RUN_SQL on schemas
- Define DLT pipeline config: cluster policy, auto-scaling, expected schema
- Include two targets: dev and prod (different workspace URLs)

Show me the full databricks.yml and deploy commands.

TAKE-HOME

You've built the stack. Now keep **building**.

What you learned today is repeatable. Genie Code is the accelerator. The data layer is yours.

Build Studio

A workspace. Notebooks. Pipelines. Deployed via Asset Bundles. Your foundation.

Public Prompts

Share prompt playbooks with your team. Genie Code learns. Consistency across teams.

Real Workflows

Describe → generate → review → deploy. Keep humans in the loop. Build fast, ship safe.

THE SHIFT

What changed from traditional ETL.

Before

- Custom Python notebooks (maintenance tax)
- Manual schema management
- Separate data lake and warehouse
- Manual governance rules
- Lengthy deployment processes

Now

- Genie Code generates and updates
- UC 3-level namespace + AI tagging
- One lakehouse (Delta + SQL)
- Policy as code (ABAC, AI Gateway)
- One-command deploy with guardrails

NEXT STEPS

Build on this. **Together.**

You have the code, the workspace, and the playbook. We're here to help you scale.

01

Keep the Workspace

This is your workspace. Keep building today's pipeline. Add your sources.

02

Connect Your Sources

Wire Kafka, Lakeflow Connect, or batch exports. Use today's Genie prompts.

03

Deploy to Production

Asset Bundles. Same repo. Multi-environment deploy. Schedule your jobs.

Build together. **Let's go.**

Kabeer Singh Thockchom
Senior Solutions Engineer

Alec Peterson
Account Executive

 **databricks**